

Laboratorio 1

Remote Debugging con VIO/ILA

Autor: Simón Saillen

Profesores: Ariel Luis Pola, Pablo Cayuela

Diseño Digital Avanzado

Fundación Fulgor / FCEfyN

Septiembre 2026

1. Introducción

Este informe tiene como objetivo documentar el desarrollo de un proyecto de diseño digital utilizando Verilog. El proyecto se centra en la implementación de un módulo TOP que integra los módulos VIO e ILA para control y depuración.

Se busca validar el comportamiento del diseño mediante un testbench y realizar la implementación en una FPGA, conectándose remotamente a la placa.

2. Objetivos

- Incluir los módulos VIO e ILA para control y debug del módulo TOP.
- Realizar un testbench que valide el comportamiento del diseño sin los IP cores VIO e ILA.
- Implementar en FPGA conectándose remotamente a la placa.
- Realizar capturas de pantalla que muestren las waveforms del Testbench e ILA.

3. Desarrollo

3.1. Material proporcionado

Se proporcionaron cinco archivos base:

- `shiftleads.v`: El modulo TOP que implementa una salida de leds ciclica.
- `shiftleads_muxVioIla.v`: El mismo modulo TOP con los modulos VIO e ILA integrados.
- `tb_shiftleads_file.v`: Testbench para el modulo TOP sin los modulos VIO e ILA.
- `Arty_Master_v2.xdc`: Archivo de constrains para los pines de la FPGA.

Los mismos fueron renombrados, cambiados y adaptados para mejor legibilidad y para cumplir con los objetivos del trabajo práctico.

3.2. Modificaciones realizadas

3.2.1. Módulo TOP.v

Se unificaron los archivos `shiftleads.v` y `shiftleads_muxVioIla.v` a `TOP.v` para tener un solo módulo que pueda ser simulado y sintetizado.

Se agregaron las instancias de los modulos VIO e ILA al final del archivo, y se agregaron los puertos de entrada y salida necesarios para controlar los modulos VIO e ILA desde el testbench.

Algunos nombres de señales fueron cambiados para mejorar la legibilidad del código.

3.2.2. Testbench `tb_TOP.v`

Se renombró el archivo `tb_shiftleds_file.v` a `tb_TOP.v`.

Sin modificar el modulo TOP, se cambió el parámetro del testbench `NB_COUNT` de 32 a 20, al hacer este cambio la cantidad necesaria de ciclos para superar el primer limite R0 pasa de 2^{22} a 2^{10} , permitiendo shifteos de registro mas seguidos y logrando una mejor observación del comportamiento.

Se corrigio el comportamiento del reset en el codigo para que actue con reset al contador luego de cada cambio de configuracion para mejorar la observacion de los cambios de las waveforms.

Tambien se agrego un ultimo estado extra en el cual se desactiva el `init` del contador para poder observar el comportamiento de los leds al inhabilitar el contador y mantener el valor de salida previo.

4. Resultados

4.1. Simulacion

Antes de modificar la cantidad de bits del contador, se realizó la simulación del testbench `tb_TOP.v` y se obtuvieron las siguientes waveforms:

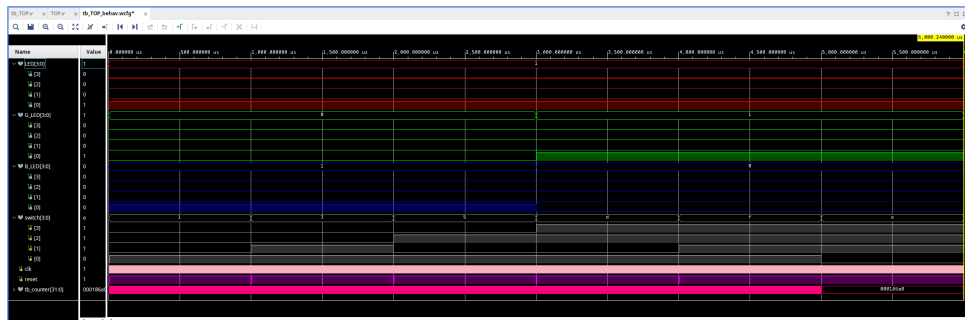


Figura 1: Simulación del testbench `tb_TOP.v` con contador de 32 bits.

Como se puede observar, el contador no logra superar el primer limite, manteniendo el valor de salida de los leds en `4'b0001` de manera constante. Pero se puede comprobar que el color del led si cambia al cambiar el valor de `switch[3]` a 1.

Una vez modificado el parámetro del contador a 20 bits, se realizó nuevamente la simulación del testbench `tb_TOP.v` y se obtuvieron las siguientes waveforms:

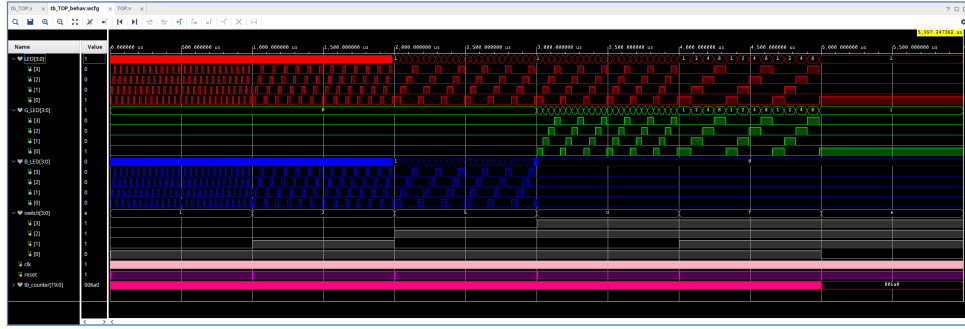


Figura 2: Simulación del testbench `tb_TOP.v` con contador de 20 bits.

Como se puede observar, el contador logra superar los distintos límites, cambiando el valor de salida de los leds de manera cíclica. Además, se puede observar el cambio de velocidad del ciclo de los leds al cambiar el valor de `SEL = switch[2:1]`. Y por último, se puede observar que al cambiar el valor de `switch[0]` a 0, se inhabilita el `init`, impidiendo que el contador pueda avanzar y manteniendo el valor de salida previo de los leds.

4.2. Implementación en FPGA

4.2.1. Conexión remota a la placa y configuración de VIO e ILA

Una vez conectado remotamente a la placa mediante SSH, se realizó la configuración de los módulos VIO e ILA para poder controlar el módulo TOP y observar su comportamiento.

El módulo VIO fue configurado para controlar las señales de entrada del módulo TOP, permitiendo cambiar los valores `select_mux`, `vio_reset` y `vio_switch` desde la interfaz de usuario.

El módulo ILA fue configurado para observar las señales de salida del módulo TOP, permitiendo capturar las waveforms de las señales `LED[3:0]`.

hw_vio_1				
Name	Value	Activity	Direction	VIO
> probe_in0_0_1[3:0] - LED	[H] 1		Input	hw_vio_1
> probe_in1_0_1[3:0] - B_LED	[H] 1		Input	hw_vio_1
> probe_in2_0_1[3:0] - G_LED	[H] 0		Input	hw_vio_1
> vio_0_probe_out2[3:0] - vio_switch	[B] 0000 ▾		Output	hw_vio_1
vio_0_probe_out0 - select_mux	[B] 1 ▾		Output	hw_vio_1
vio_0_probe_out1 - vio_reset	[B] 0 ▾		Output	hw_vio_1

Figura 3: Configuración del módulo VIO en la FPGA.

4.2.2. Captura de waveforms con ILA

Se realizaron distintos escenarios de prueba utilizando el módulo VIO para cambiar los valores de entrada del módulo TOP y se capturaron las waveforms correspondientes con el módulo ILA.

Escenario 1: Immediate (Idle)

Se configuraron los valores de `select_mux = 1`, `vio_reset = 1` y `vio_switch = 4'b1001`.

ILA con trigger en modo immediate, capturando las señales de salida de los leds en el instante de captura.

Se lograron capturar las siguientes waveforms:

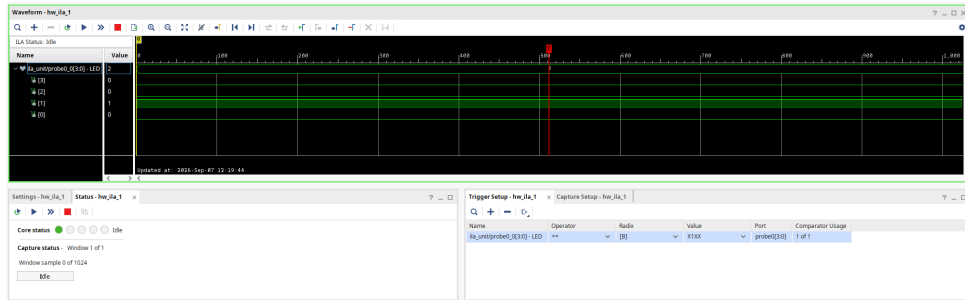


Figura 4: Captura de waveforms con ILA en estado immediate (Idle).

Como se puede observar, la señal capturada muestra el estado actual de los leds en el instante de la captura, ignorando el trigger seteado en el ILA.

Escenario 2: Desplazamiento Ciclico

Se configuraron los valores de `select_mux = 1`, `vio_reset = 1` y `vio_switch = 4'b1001`. Habilitado, con velocidad R0 y leds verdes activos.

Debido al limitante de cantidad de muestras del ILA, se tuvieron que realizar varias capturas para poder observar el comportamiento completo del desplazamiento cíclico de los leds.

Cambiando el valor del trigger del ILA:

- Primera captura: `LED == 4'bXX1X`
- Segunda captura: `LED == 4'bX1XX`
- Tercera captura: `LED == 4'b1XXX`
- Cuarta captura: `LED == 4'bXXX1`

Las capturas de waveforms obtenidas fueron las siguientes:

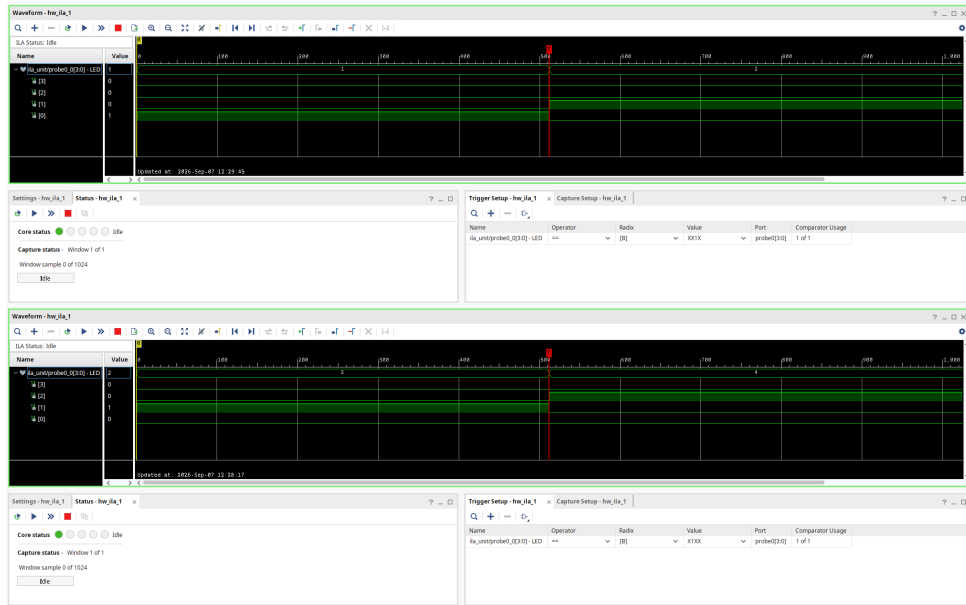


Figura 5: Captura de waveforms con ILA en estado desplazamiento cíclico (1).

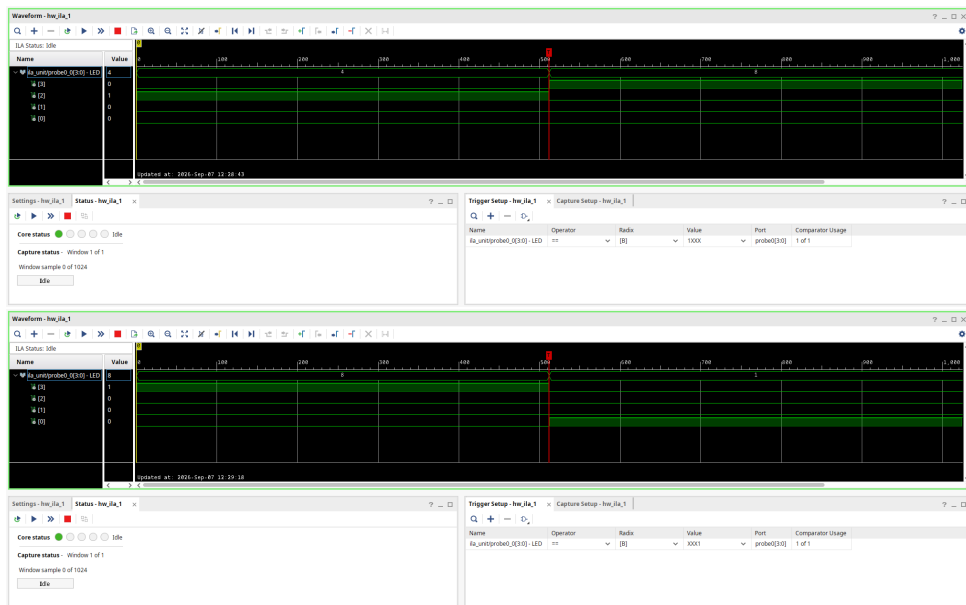


Figura 6: Captura de waveforms con ILA en estado desplazamiento cíclico (2).

Juntando las cuatro capturas, se puede observar el desplazamiento cíclico de los leds en el orden correcto, cumpliendo con el comportamiento esperado del modulo TOP.

Escenario 3: Control de Color de Leds

Se configuraron los valores de `select_mux = 1`, `vio_reset = 1` y `vio_switch = 4'b1001`. Habilitado, con velocidad R0 y leds verdes activos.

Para comprobar el cambio de color de los leds no se puede observar directamente en las waveforms ya que el ILA solo captura el estado de los leds generales LED, por eso hay que observar los cambios en VIO.

Para observar el cambio de color se cambio el valor de `vio_switch[3] = 0`, cambiando el color de los leds de verde a azul.

Esto es observable en la siguiente captura de pantalla, donde se ve reflejado que leds estan activos mediante las flechas de actividad del VIO:

hw_vio_1

Q [Zoom In] [Zoom Out] [Reset] [Add] [Remove]

Name	Value	Activity	Direction	VIO
> probe_in0_0_1[3:0] - LED	[H] 2		Input	hw_vio_1
> probe_in1_0_1[3:0] - B_LED	[H] 0		Input	hw_vio_1
> probe_in2_0_1[3:0] - G_LED	[H] 2		Input	hw_vio_1
> vio_0_probe_out2[3:0] - vio_switch	[B] 1001		Output	hw_vio_1
vio_0_probe_out0 - select_mux	[B] 1		Output	hw_vio_1
vio_0_probe_out1 - vio_reset	[B] 1		Output	hw_vio_1

hw_vio_1

Q [Zoom In] [Zoom Out] [Reset] [Add] [Remove]

Name	Value	Activity	Direction	VIO
> probe_in0_0_1[3:0] - LED	[H] 2		Input	hw_vio_1
> probe_in1_0_1[3:0] - B_LED	[H] 2		Input	hw_vio_1
> probe_in2_0_1[3:0] - G_LED	[H] 0		Input	hw_vio_1
> vio_0_probe_out2[3:0] - vio_switch	[B] 0001		Output	hw_vio_1
vio_0_probe_out0 - select_mux	[B] 1		Output	hw_vio_1
vio_0_probe_out1 - vio_reset	[B] 1		Output	hw_vio_1

Figura 7: Captura de los probes del VIO pasando de salida de leds verde a azul.

Escenario 4: Inhabilitación de Contador

Se configuraron los valores de `select_mux = 1`, `vio_reset = 1` y `vio_switch = 4'b1001`. Habilitado, con velocidad R0 y leds verdes activos.

Para comprobar esto se cambio el valor de `vio_switch[0] = 0`, inhabilitando el contador y manteniendo el valor de salida de los leds previo al cambio.

Luego de ejecutar el trigger immediate se obtuvo la siguiente waveform:

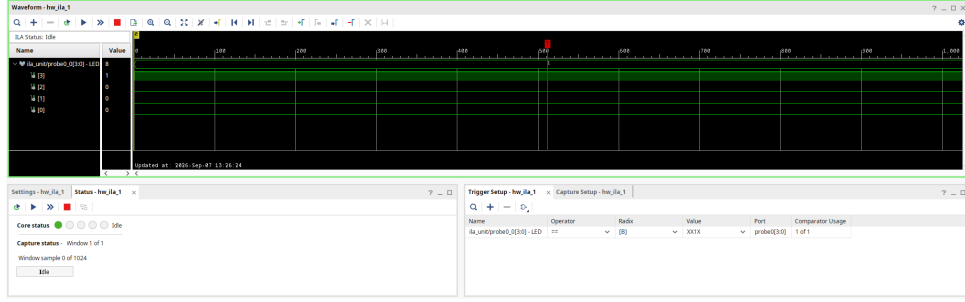


Figura 8: Captura de waveforms con ILA en estado inhabilitación de contador.

Este valor de salida en teoría está manteniéndose constante, para comprobar esto se cambia el valor del trigger del ILA a un valor distinto, por ejemplo `LED == 4'bX1XX` y se debería observar que el trigger nunca se cumple, quedando en `waiting for trigger`.

Como se predecía, el trigger nunca se cumple y el ILA queda en estado de espera:

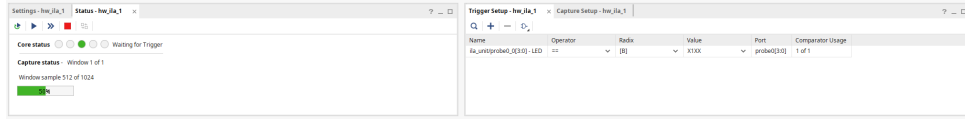


Figura 9: ILA en estado de espera.

5. Conclusiones

A partir de los escenarios ensayados, se validó de manera exitosa el funcionamiento del módulo TOP sobre la FPGA utilizando un entorno de depuración remota. Mediante el módulo VIO se logró la inyección de estímulos virtuales interactivos, emulando con precisión la periferia de switches y botones físicos de la placa. Por el otro, el módulo ILA posibilitó la inspección temporal interna de señales de alta velocidad directas del reloj de 100 MHz.

El análisis de las waveforms obtenidas ratificó el correcto comportamiento secuencial del diseño. A pesar de la restricción en la profundidad de memoria del buffer del ILA (1024 muestras básicos), el uso estratégico de máscaras de bit con estados de indiferencia `X` permitió reconstruir secuencialmente el desplazamiento circular completo del registro en el orden correcto. Asimismo, las pruebas estáticas de inhabilitación mediante la señal `init` comprobaron empíricamente la retención de estado de los flip-flops, evidenciada por el bloqueo en estado *Waiting for Trigger* del analizador lógico.

Se verificó la correcta segregación de las herramientas de diagnóstico: mientras que los eventos de naturaleza secuencial y temporal dependientes del reloj fueron auditados con precisión mediante el ILA, la lógica puramente combinatorial encargada del control de color del periférico (`G_LED` y `B_LED`) fue satisfactoriamente comprobada a nivel general mediante el monitoreo directo de *probes* en el VIO Dashboard.

Como balance final, el flujo completo de diseño y verificación (desde la generación de vectores de prueba en simulación hasta la ejecución e inspección en hardware real) demostró ser robusto, funcional y alineado con las especificaciones de diseño requeridas.